ThreadLearn: A RAG-Augmented Fine-Tuned Language Model for JavaScript Concurrency Bug Detection and Remediation

Le Tri Trung[1] [0009-0007-0790-1388], Ha Van An[2] [0009-0000-8536-4396], and Nguyen Thi Thuy Hoai[3] [0009-0007-6328-0715]

[1] FPT University, Da Nang, Vietnam
letritrung2605@gmail.com

[2] FPT University, Da Nang, Vietnam
van.an.webdev@gmail.com

[3] FPT University, Da Nang, Vietnam
hoaintt40@fe.edu.vn

Abstract. Concurrency bugs in asynchronous JavaScript (Node.js) code are a leading cause of serious failures in production systems, including data races, lost updates, and application hangs. Existing tools fall into two categories: rule-based static detectors that can identify suspicious patterns but cannot generate fixes, and large language models (LLMs) that can reason about code but tend to miss the exact buggy line and require large model sizes (7B–32B parameters) to perform well. ThreadLearn addresses both weaknesses by combining a 5-pattern static race detector with a small language model (Qwen2.5-Coder-1.5B) fine-tuned using QLoRA on 892 training examples with hindsight Chain-of-Thought reasoning. At inference time, a BM25 retrieval pipeline fetches the 3 most relevant documents from a 2,050-document knowledge base to augment the prompt. On a 30-case real-world benchmark drawn entirely from production npm packages spanning 10 concurrency bug categories, ThreadLearn with the full BM25+AST pipeline achieves 22.0/30 (73.3%), compared with 60.0% for the base model and 65.0% for GPT-3.5-turbo with the same pipeline, despite having ~125× fewer parameters. A key finding is that the retrieval pipeline only benefits models that already have domain knowledge.

Keywords: Concurrency bugs · Race conditions · Static analysis · LLM fine-tuning · Retrieval-augmented generation · JavaScript.

1 Introduction

Asynchronous JavaScript (Node.js [13]) powers web backends, APIs, and real-time services, but its event-driven, non-blocking model invites subtle concurrency bugs. A large-scale empirical study (NodeCB [1]) found that 93% of 57 real Node.js concurrency bugs cause crashes, corruption, or hangs.

Despite how common and harmful these bugs are, existing tools still fall short. Rule-based static analyzers (ESLint async, ThreadSanitizer) detect suspicious patterns but cannot fix them, and they generate false alarms on modern async/await code. LLM-based approaches [2] reason about code semantics but cannot pinpoint the buggy line without structural guidance and require large models for competitive accuracy.

Four gaps drive this work: (G1) rule-based detectors find bugs but cannot fix; (G2) LLM-only approaches ignore static-structure signals and need 7B–32B models; (G3) no existing tool feeds detector output as structured context to an LLM; (G4) no tool covers detect→explain→fix end-to-end for JavaScript concurrency bugs.

We structure the evaluation around three research questions:

RQ1. Does combining a static race detector with a fine-tuned small language model outperform a general-purpose LLM (GPT-3.5-turbo) on real-world JavaScript concurrency bugs, despite far fewer parameters? (Sect. 5)

RQ2. Does BM25+AST retrieval benefit every model equally, or does its benefit depend on whether the model has been fine-tuned on domain data? (Sect. 5)

RQ3. Which concurrency bug categories remain hard for a fine-tuned small model, and what does that reveal about the limits of pattern-based training data? (Sect. 5)

The remainder of this paper is organized as follows. Section 2 provides background on JavaScript concurrency bug taxonomy and the core techniques used. Section 3 surveys related work across rule-based detectors and LLM approaches. Section 4 describes the dataset construction, static detector design, fine-tuning procedure, and end-to-end pipeline. Section 5 reports experimental results answering RQ1–RQ3. Section 6 discusses implications, answers research questions, and highlights the privacy advantage of local deployment. Section 7 concludes and outlines future research directions.

2 Background

Concurrency bug taxonomy. Concurrency bugs in asynchronous JavaScript arise from the single-threaded event loop model: a single thread interleaves callback execution, I/O completion, and timer firing in an order that depends on runtime scheduling rather than the order code is written. Unlike race conditions in multi-threaded languages (unsynchronized memory access across cores), Node.js bugs stem from interleaving async operations whose completion order is not guaranteed. The NodeCB study [1] identifies three recurring types: atomicity violations (65%), where two logically atomic steps (e.g., check-then-update) execute as separate interruptible operations because Node.js encourages fire-and-forget I/O; order violations (30%), where a callback assumes a prior async step completed; and starvation (5%), where an exhausted I/O queue leaves deferred tasks unexecuted. We add a fourth, JavaScript-specific type—closure loop var—where function-scoped var makes a loop variable

referenced inside a deferred callback observe its final value rather than the value at scheduling time; though syntactic, it is frequent in our benchmark and fully fixable by a deterministic rewrite (var→let).

BM25 retrieval. BM25 [6] scores by term frequency and inverse document frequency with length normalization (k1=1.5, b=0.75); we use it over dense retrieval because exact API name matching (e.g., setTimeout) is more precise for concurrency patterns.

QLoRA fine-tuning. QLoRA [3] fine-tunes a 4-bit quantized model via LoRA [10] adapters, updating only adapters so training fits a single laptop GPU (r=16, α=32).

3 Related Work

Rule-based detectors match code against predefined patterns. ThreadSanitizer [4] detects data races at runtime but only on C/C++ and Java; ESLint [11] adds async rules for JavaScript but only syntactic ones, missing semantic races across callbacks; the NodeCB regex matcher [1] has high false-positive rates on modern async/await code; and dynamic delay-injection (NACD [5]) exposes more event races but still generates no fixes. ML/LLM approaches train on code features for defect detection, but pre-trained code models are not concurrency-specific; PCWMs [2] applies reasoning world models to parallel code yet relies on 7B–32B models and ignores static structure; and RAG code tools [7] improve completion but have not been applied to JavaScript concurrency remediation. Table 1 positions ThreadLearn against these tools along four axes.

Fig. 2. Comparison of ThreadLearn with related tools. ✓ = supported, X = not supported.

Tool	Detect	Fix	JS	Fine-tuned
ThreadSanitizer	X	X	X	X
ESLint async	✓	X	✓	X
NodeCB (rules)	✓	X	✓	X
PCWMs (LLM-only)	✓	✓	✓	✓
ThreadLearn (This work)	✓	✓	✓	✓

The rule-based tools flag a line without fixing it; PCWMs generates fixes but without structural signal, requiring 7B-32B parameters. ThreadLearn is the only entry combining all four: its detector narrows search to pattern_id and line_range before LLM inference, letting a 1.5B model match a far larger one (Sect. 5).

4 Proposed Approach

4.1 Dataset

Our study uses three datasets. The fine-tuning dataset (892 examples) has the form (code, detector_output, reasoning_trace, fix), where each reasoning trace is written by a teacher model after the correct fix is known (hindsight CoT), and was built entirely by the authors with no crowdsourcing or model-generated labels. It comprises 332 synthetic pairs (37.2%)—handcrafted and manually verified—and 560 generated pairs (62.8%) produced by generate_pairs(), a template engine that expands the handcrafted pairs over 40 domain slots (e.g., User, Order) using 18 generator functions, each a distinct concurrency transformation (e.g., sequential await→Promise.all, callback→async, unguarded fetch→AbortController timeout). Because each generator is a deterministic template, all generated labels are correct by construction, avoiding the label noise of LLM-sampled data.

The knowledge base contains 2,050 JavaScript documents from authoritative sources: MDN Web Docs [15] (99 docs), concurrency-library docs (RxJS, p-limit, async-mutex, Bluebird; 50 docs), curated ThreadLearn pattern descriptions (151 docs), and 1,750 synthetic documents via context permutation. All fall into three groups: patterns (1,418), race-conditions (352), anti-patterns (280).

The real-world benchmark has 30 JavaScript concurrency bugs from production open-source packages, spanning 10 categories: Race Condition (5), Unhandled Rejection (5), Double Callback (4), Resource Exhaustion (4), Event Loop Blocking (3), Sequential Awaits (3), Zalgo (2), Context Loss (2), Stream Leak (1), and Callback Hell (1), from production npm packages (request, mysql, express, etc.). Every case (rw_01–rw_30) traces to a specific GitHub issue or Node.js API document (e.g., request#2484, mysqljs#1260), so no case is synthetic; drawing them from independent production incidents rather than the training-data engine eliminates model-favorable bias.

4.2 Architecture and Static Detector

AST preprocessing. ThreadLearn's preprocessor provides four esprima-based [9] helpers: stripComments and normalizeWhitespace canonicalize the source so the detector's regex patterns and BM25 query are not thrown off by formatting; extractFunctions scopes detector line-ranges to a function body; and extract_keywords returns the top-20 identifier names as the literal BM25 query.

Evaluation strategy. Each test case specifies an expected fix pattern (e.g., Promise.all, try/catch, let in loop) and is scored PASS if the generated code contains it, PARTIAL if code is generated without the pattern, and FAIL if no code is generated.

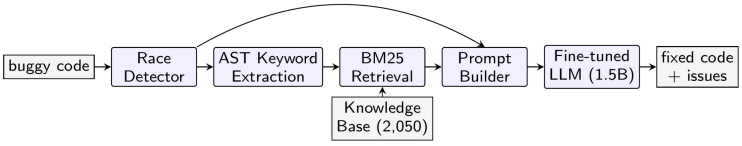

Fig. 1. ThreadLearn end-to-end pipeline. Buggy JavaScript code flows through five modules: static race detection, AST keyword extraction, BM25 retrieval from a 2,050-document knowledge base, prompt construction, and fine-tuned LLM inference, producing a diagnosis and corrected fix.

The pipeline (Fig. 1) runs five steps: (1) static detection—race_detector scans the source in O(n) time, emitting {pattern_id, line_range, description} per match; (2) keyword extraction—extract_keywords() walks the esprima AST, keeping CamelCase API names intact; (3) retrieval—BM25 returns the top-3 of 2,050 documents; (4) prompt construction—prompt_builder assembles detector report, retrieved docs, and buggy code; (5) fix generation—the fine-tuned LLM outputs corrected code with reasoning.

Static race detector. The detector checks 5 patterns in two groups. Closure/async: closure_loop_var (var captured by reference in loop callbacks), shared_var_settimeout (shared variable modified inside setTimeout), promise_no_await (Promise created but not awaited). Shared state: concurrent_write_array (array mutated from concurrent callbacks) and counter_no_atomic (non-atomic counter increment). For example, closure_loop_var fires when a var loop variable is captured inside a setTimeout or Promise callback, so all callbacks read the final loop value; the fix replaces var with let:

5 Experimental Results

All experiments use the 30-case benchmark (Sect. 4.1), run on a Kaggle T4 GPU for local models and the OpenAI API for GPT-3.5-turbo [14]. We evaluate six configurations—{base, fine-tuned, GPT-3.5-turbo} × {no-pipeline, +pipeline}, where the pipeline is AST keyword extraction → BM25 top-3 retrieval → augmented prompt—scored PASS=1.0, PARTIAL=0.5, FAIL=0.

This experiment tests whether combining static detection with fine-tuned small-LLM inference and BM25+AST retrieval outperforms separate rule-based detection and general-purpose LLM baselines. The 2×3 design separates fine-tuning contribution from pipeline contribution.

Fig. 3. Full ablation on the 30-case real-world benchmark (score = PASS + 0.5xPARTIAL).

Configuration	Pass	Partial	Fail	Score	%
Base (no fine-tune, no pipeline)	6	24	0	18.0	60.0%
GPT-3.5-turbo (no pipeline)	7	23	0	18.5	61.7%
Fine-tuned only (no pipeline)	8	22	0	19.0	63.3%
Base + pipeline	8	20	2	18.0	60.0%
GPT-3.5-turbo + pipeline	9	21	0	19.5	65.0%
ThreadLearn + pipeline	14	16	0	22.0	73.3%

Fine-tuning contribution (no pipeline): +1.0 pt (+3.3 pp). Pipeline contribution (fine-tuned): +3.0 pt (+10.0 pp). Combined gain vs. base: +4.0 pt (+13.3 pp).

Without any pipeline, the three models score in a narrow band (60.0–63.3%) with zero FAILs: every model emits syntactically valid code, and the dominant failure is surface-level rewriting into async/await without fixing the hazard. ThreadLearn already edges out GPT-3.5-turbo here (+0.5 pt), confirming G2. Adding BM25+AST then separates the models sharply: the fine-tuned model gains +3.0 pts (+10 pp), GPT-3.5-turbo only +1.0 pt, and the base model nothing (60.0% in both) while adding 2 FAILs.

6 Discussion

Three implications follow. (i) Fine-tuning is a prerequisite for pipeline benefit—the base model cannot use retrieved docs. (ii) The pipeline amplifies fine-tuning (+3.0 pts on top of fine-tuning's +1.0, 3x larger). (iii) ThreadLearn (73.3%) beats GPT-3.5-turbo+pipeline (65.0%) by +2.5 pts despite ~125x fewer parameters.

Answering RQ1 and RQ2. The third point answers RQ1: the static detector's structured output narrows what the LLM must infer, so a fine-tuned 1.5B model beats GPT-3.5-turbo. The first answers RQ2: retrieval does not benefit every model equally—+10 pp for the fine-tuned model versus 0 pp (and regression) for the base model—so retrieval only pays off once a model already has domain knowledge.

Per-category analysis and RQ3. Breaking the 22.0/30 down by category, ThreadLearn is strongest on Unhandled Rejection (90%), Sequential Awaits (100%), and Race Condition (70%, up from 50% for both baselines)—categories tied to specific fix templates. It is weakest on Zalgo, Double Callback, and Stream Leak (all

50%), the last being the only category where it trails the base model. This answers RQ3: these remain hard because their fixes require tracing callback invocation order across asynchronous boundaries—a semantic property not inferable from local syntactic context—whereas our template-generated training data is dominated by syntactic substitutions. Pattern-template data thus generalizes well to syntactically-fixable bugs but underrepresents bugs needing multi-step temporal reasoning.

This paper makes four contributions:

1. race_detector: a 5-pattern static analyzer for JavaScript (Node.js) with an AST-based code preprocessor (Sect. 4.2).

2. Hindsight CoT dataset: 892 training examples of the form (code, detector_output, reasoning_trace, fix), where reasoning is written after the correct fix is known (Sect. 4.3).

3. ThreadLearn pipeline: an end-to-end system that detects, retrieves context, and generates a fix, served via a FastAPI web endpoint (Sect. 4.2).

4. Evaluation: comparison against a base LLM and GPT-3.5-turbo on a 30-case real-world benchmark across fix pass rate, per-category accuracy, and ablation (Sect. 5).

Privacy advantage of local deployment. ThreadLearn uses Qwen2.5-Coder-1.5B—a small open-weight model that runs entirely on-premise without sending source code to any external API. Cloud-based services (GPT-3.5-turbo, GPT-4, Claude) transmit proprietary code over the network to third-party infrastructure, risking data leakage, IP exposure, or compliance violations under GDPR, HIPAA, or corporate policies. For enterprises handling sensitive codebases, ThreadLearn delivers superior fix accuracy (73.3% vs. 65.0%) while keeping all data within the organization's boundary—and its 1.5B parameter footprint runs on a single T4 GPU, requiring no expensive clusters or API subscriptions.

7 Conclusion and Research Directions

ThreadLearn addresses the gap between static detection and automated remediation by combining a 5-pattern rule-based race detector with a small language model (Qwen2.5-Coder-1.5B) fine-tuned on 892 hindsight Chain-of-Thought examples and a BM25+AST retrieval pipeline. On a 30-case real-world benchmark from production npm packages, ThreadLearn achieves 22.0/30 (73.3%)—a +13.3 pp gain over the base model and +8.3 pp over GPT-3.5-turbo with the same pipeline, despite ~125× fewer parameters. It is the first end-to-end system to pair a rule-based race detector with a fine-tuned small language model and RAG for JavaScript concurrency bug remediation, bridging PCWMs [2] (LLM-only, large model) and rule-based detectors (ESLint, ThreadSanitizer) that produce no fixes. The ablation establishes a design principle: retrieval only helps models that already have domain knowledge, so pipeline augmentation should be paired with domain-specific fine-tuning.

Future work will extend the detector to TypeScript AST patterns, add mutex/semaphore patterns, and test whether a larger fine-tuned model (e.g., Qwen2.5-Coder-7B) pushes the hard categories above 75%.

References

1. Wang, J., Dou, W., Gao, Y., Gao, C., Qin, F., Yin, K., Wei, J.: A comprehensive study on real world concurrency bugs in Node.js. In: Proc. 32nd IEEE/ACM ASE, pp. 520–531 (2017).
2. Singh, G., Guha, A., Kailkhura, B., Menon, H.: Learning reasoning world models for parallel code. arXiv:2604.20926 (2026).
3. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: efficient finetuning of quantized LLMs. In: Proc. NeurIPS (2023).
4. Serebryany, K., Iskhodzhanov, T.: ThreadSanitizer: data race detection in practice. In: Proc. WBIA, pp. 62–71 (2009).
5. Endo, A.T., Møller, A.: Event race detection for Node.js using delay injections. In: Proc. 39th ECOOP, pp. 9:1–9:28 (2025).
6. Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. Found. Trends Inf. Retr. 3(4), 333–389 (2009).
7. Lewis, P., Perez, E., Piktus, A., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proc. NeurIPS (2020).
8. Qwen Team: Qwen2.5-Coder technical report. arXiv:2409.12186 (2024).
9. Hidayat, A.: Esprima: ECMAScript parsing infrastructure. https://esprima.org, last accessed 2026/06/30.
10. Hu, E.J., Shen, Y., Wallis, P., et al.: LoRA: low-rank adaptation of large language models. In: Proc. ICLR (2022).
11. Zakas, N.C.: ESLint: the pluggable JavaScript linter. https://eslint.org, last accessed 2026/06/30.
12. von Werra, L., Belkada, Y., Tunstall, L., et al.: TRL: transformer reinforcement learning. https://github.com/huggingface/trl, last accessed 2026/06/30.
13. Dahl, R.: Node.js: evented I/O for V8 JavaScript. https://nodejs.org, last accessed 2026/06/30.
14. OpenAI: GPT-3.5 Turbo. https://platform.openai.com/docs/models, last accessed 2026/06/30.
15. Mozilla: MDN Web Docs: JavaScript reference. https://developer.mozilla.org/en-US/docs/Web/JavaScript, last accessed 2026/06/30.
16. Wei, J., Wang, X., Schuurmans, D., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: Proc. NeurIPS (2022).
17. Brown, D.: rank-bm25: a collection of BM25 algorithms in Python. https://github.com/dorianbrown/rank_bm25, last accessed 2026/06/30.